

Claude

Thursday, October 8, 2026 · CLAUDE

This is the entire code base of a small broken python game. Which files do I need to change how in order to fix it? Give me the corrected files

I'll write the fixed files, covering the bug (Task 1) and the three features (Tasks 2-4).

I fixed the bug and built all three features. A quick headless test confirmed that a coin scores once, the game ends, and R resets everything.

The bug (Task 1)

The bug is in `game/game_engine.py`. `update()` added the score for every coin the player overlapped, but never removed the coin. The player's rectangle overlapped it for many frames, so it scored again every frame.

The fix is to remove each coin the moment it's collected. I also spawn a replacement so the field stays full for the 30-second round:

python

```
for coin in check_collection(self.player, self.coins):
    self.score += coin.value
    self.coins.remove(coin)
    self.coins.append(self._random_coin())
```

Files changed

- `game/game_engine.py`: Contains the bug fix above. It also adds lives (3), a 30-second timer, obstacle updates, a game-over state, and a `reset()` method. The R key calls `reset()` once the round is over.
- `game/coin.py`: Coins now have a type, defined in a `COIN_TYPES` table. Bronze is worth 1 point, silver 3 and gold 5, each with its own color. Bronze spawns most often and gold least.
- `game/obstacle.py` (**new file**): Red squares that bounce around the play area. They are clamped to the screen, so they never leave it.
- `game/collection.py`: Adds `check_obstacle_hit()` alongside the existing `check_collection()`.
- `game/renderer.py`: Draws the obstacles, makes the player flash white after a hit, and adds a game-over overlay with the final score and "Press R to play again".

Obstacle hits

A hit costs one life and starts a 1.5-second grace period, during which the player flashes. This stops a single touch from draining a life every frame, which the README asks for. The round ends when the timer reaches 0 or lives reach 0.

`game/__init__.py`, `game/player.py` and `main.py` are unchanged. Drop the new files into your `coin-collector/game/` folder, add `obstacle.py` there, and run `python main.py`.

For your submission, a "before" video of the original code will show the score racing upward while you stand on a coin. An "after" video can show the three coin colors, a life being lost, and the game-over screen followed by R to restart.

Game engine

PY

Coin

PY

Obstacle

PY

Collection

PY

Renderer

PY



Ok I need to fix and update each bug individually and push to my repo. Could you break it down into say 3 phases of bug fixes and what commit message I should put to ? Try to make each bug fix limited to changing a single file if possible.

Splitting this cleanly takes 8 small commits grouped into 4 phases, so I'm going beyond 3 phases. Phases 3 and 4 touch several files. I'll build and test each phase's files so every commit leaves the game runnable.

I split the work into 4 phases (8 commits), not 3, so that each commit changes one file wherever possible. I applied the phases in order to a copy of your original code and the game ran after each one, with coins scoring once.

Each phase has its own folder (`phase1/` to `phase4/`) containing only the files that change. Copy them over `game/` in your repo, then commit.

Phase 1: Fix the double-scoring bug (1 file)

Commit 1 copies `phase1/game_engine.py`.

```
fix: remove coin after collection so it scores only once
```

Phase 2: Coin types (2 files)

This can't be a single file, because `coin.py` defines the types and `game_engine.py` picks them. Commit them in this order, or the game will crash between commits.

Commit 2 copies `phase2/coin.py`. It's backward compatible, so the game still runs.

```
feat(coin): add bronze/silver/gold coin types with values and colors
```

Commit 3 copies `phase2/game_engine.py`.

```
feat(engine): spawn weighted random coin types
```

Phase 3: Obstacles and lives (4 files, 4 commits)

The first three commits only add code, so the game behaves the same until the engine uses it in commit 7.

Commit 4 adds `phase3/obstacle.py`.

```
feat(obstacle): add bouncing Obstacle entity clamped to play area
```

Commit 5 copies `phase3/collection.py`.

```
feat(collection): add check_obstacle_hit
```

Commit 6 copies `phase3/renderer.py`.

```
feat(renderer): draw obstacles and flash player when hit
```

Commit 7 copies `phase3/game_engine.py`.

```
feat(engine): add obstacles, 3 lives, and hit grace period
```

After this commit the game freezes with a "GAME OVER" banner when lives hit 0, and there's no restart yet.

Phase 4: Timer and restart (2 files)

Commit 8 copies `phase4/renderer.py`.

```
feat(renderer): add game-over overlay with final score and restart prompt
```

Commit 9 copies `phase4/game_engine.py`.

```
feat(engine): add 30s round timer, game over, and R to restart
```

That's 9 commits in total. Phase 3 is four commits because it adds four files. If you'd rather have exactly 3 phases, merge phase 4 into phase 3.

Two things to check before you push:

- `game/obstacle.py` is new, so run `git add game/obstacle.py` for commit 4.
- Your folder tree shows `requirements.txt`, so make sure it contains `pygame`.

Game engine

PY

Coin

PY

Game engine

PY

Obstacle

PY

Collection

PY

Renderer

PY

Game engine

PY

Renderer

PY

Game engine

PY

